

ALGORAND

A Secure and Efficient Distributed Ledger

Jing Chen · Silvio Micali

arXiv 1607.01341v9 · May 2017

Blockchain Class · Paper Presentation

Agenda

Roadmap for the next ~70 minutes

- **1. Context, Who, When, Why another blockchain?**
- **2. Consensus, Pure PoS + Byzantine Agreement (the heart of the paper)**
 - VRFs · cryptographic sortition · committees · BA* · safety vs liveness
- **3. Execution, Algorand Virtual Machine (AVM) + TEAL**
 - vs Bitcoin Script · vs EVM, why "non-Turing-complete" is a feature, not a bug
- **4. State model, accounts, finality, fast catchup, participation keys**
- **5. Trade-offs and criticisms**
- **6. TL;DR and Q&A**

Who wrote this, and why should we care?

A Turing-award cryptographer decides to fix blockchain

Silvio Micali

MIT · CSAIL

Turing Award (2012), with Goldwasser, for foundations of modern cryptography.

Co-invented:

- Probabilistic encryption
- Zero-knowledge proofs
- Verifiable Random Functions (VRFs), 1999
- Secure multi-party computation

Translation: when Micali says "I've built a better BFT", people pay attention.

Jing Chen

Stony Brook University

Algorithmic game theory, mechanism design.

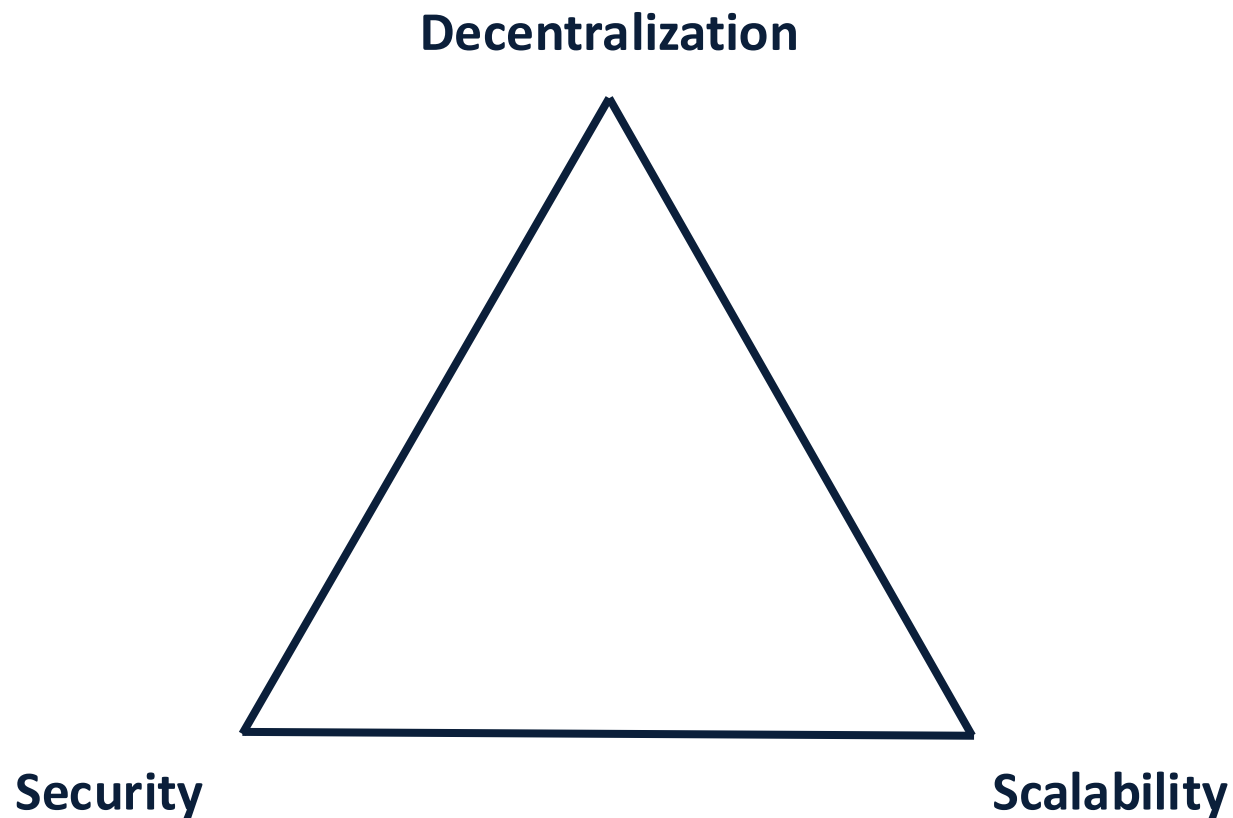
Timeline:

- 2016, first arXiv preprint
- May 2017, v9 (what we're reading)
- 2017, Algorand Inc. founded
- June 2019, Algorand mainnet launches
- 2020+, AVM, smart contracts, ASAs

The paper is 75 pages. We're reading the ideas, not the proofs.

The framing: the Blockchain Trilemma

Pick two, supposedly. Algorand claims all three.



We'll see how, keep this triangle in mind.

Where do we land today?

Bitcoin → decentralized + secure, **slow** (~7 tps, 60-min finality)

Ethereum → decentralized + secure, **L1 still slow** (~15 tps)

Classical PBFT → fast + safe, **not decentralized** ($O(n^2)$ msgs → ~100 nodes max)

Algorand's pitch: all three, via **tiny random committees** sampled cryptographically from **all stakers**.

So... why another blockchain?

You know PoW, PoS, and PBFT. Here's what's broken with each.

Bitcoin (PoW)

- Energy $\approx$ Argentina
- ~ 7 tps, 10-min blocks
- Probabilistic finality (6 confirmations)
- Miner centralization (3-4 pools $> 50\%$)
- 51% attack is a real line

Ethereum (PoS)

- ~ 15 tps on L1
- Finality in ~ 12.8 min (2 epochs)
- Nothing-at-stake solved via slashing
- But: long-range attacks still messy

PBFT / HotStuff

- $> 2/3$ honest, safe + live
- $O(n^2)$ message complexity
- $\rightarrow$ doesn't scale past ~ 100 nodes
- Not permissionless (who's in the set?)
- Great for Hyperledger, bad for "public"

Algorand's thesis: combine the honesty guarantees of BFT with the openness of PoS, without $O(n^2)$ messages.

Bitcoin vs Algorand:



Algorand in four big ideas

Preview, we'll unpack each one

1

Pure Proof-of-Stake

Every algo held = one "coin-vote".
No staking, no delegation, no lock-up.
Holding the token = participating.

2

Cryptographic Sortition (VRF)

At each step, a tiny committee (~1000–2000)
is secretly, randomly picked from all users,
weighted by stake. You find out if YOU're on
it.

3

BA*, Message-Passing Byzantine Agreement

A new BFT that runs on the committee,
not on everyone. $O(1)$ communication per
user.
Guaranteed: no forks under normal
network.

4

Player Replacement

Each step uses a fresh committee
→ even if adversary corrupts members after
they speak,
their work is already done. Adaptive-secure.

PART 1

Consensus

How does a random group of strangers agree on what happened?

Consensus recap, one slide

You know this. Stay with me for 30 seconds.

Mechanism	Who proposes?	Who agrees?	Cost	Finality
PoW (BTC)	Winner of hash lottery	Longest-chain rule	Electricity	Probabilistic
PoS (ETH)	Randomly-chosen validator	All active validators	Stake	After 2 epochs (~13 min)
PBFT	Primary (round-robin)	> 2/3 of a fixed set	Messages (n^2)	Immediate
Algorand	Sortition winner (VRF)	> 2/3 of a random committee	Stake + a bit of compute	Immediate (no forks)

The interesting row is the bottom one, everything else is review.

The core problem Algorand is solving

"How do I run BFT when I have millions of nodes?"

PBFT scaling wall

$O(n^2)$ messages per decision.

With $n = 1,000,000$ users $\rightarrow 10^{12}$ messages.

Even at 1 KB each, that's a petabyte per block.

Classical PoS pitfalls

- Nothing-at-stake, no cost to equivocate
- Long-range attack, buy old keys cheap
- Predictable leaders $\rightarrow$ DoS the next one

Algorand's answer (in 3 moves)

1. Don't run BFT on everyone, pick a small random committee each step.
2. Pick it **cryptographically and secretly**, adversary doesn't know who to bribe or DoS until they speak.
3. Once they speak, **replace them**, corrupting them after the fact buys nothing.

Building block: Verifiable Random Function (VRF)

A hash function, but signed and uniquely verifiable, Micali 1999.

Intuition

Regular hash: $H(\text{seed}) \rightarrow y$, anyone who has the seed can reproduce y .

VRF: $VRF_{sk}(\text{seed}) \rightarrow (y, \pi)$, only holder of sk can compute y , but anyone with pk and π can **verify** that y was honestly derived.

Three properties

Uniqueness, for a given sk and seed, only one valid (y, π) exists.

Pseudorandomness, to anyone without sk , y is computationally indistinguishable from random.

Verifiability, given $(pk, \text{seed}, y, \pi)$, anyone can check correctness in $O(1)$.

Think of it as: "a coin flip you can prove you flipped fairly, without revealing the coin beforehand."

VRF, a worked example

"Am I on the committee this round?"

```
// Every user, every step:

seed  = Q_r           // round-r random
beacon
(y,  $\pi$ ) = VRF_sk(seed || role)

// Am I in the committee?
p  =  $\tau$  / W           //  $\tau$ =target size,
W=total stake
j  = Binomial( stake_i, p, y )

if j > 0:
    broadcast( msg,  $\pi$ , j )
    // j "sub-votes", proportional to stake
```

This is "cryptographic sortition", Herlihy's suggestion, per §1 of the paper.

What this buys us

- Each user checks locally, no communication to "apply" to committee.
- Expected committee size = τ (e.g., 1000–2000), actually small deviation.
- Weighted by stake, bigger stake, more sub-votes (Pure PoS).
- Adversary can't predict the committee in advance.
- When a committee member broadcasts (msg, π , j), everyone verifies π locally.
- No "selection round", no registration, no Merkle proofs of eligibility.

VRF, under the hood

Algorand uses ECVRF, elliptic-curve VRF, RFC 9381 / Goldberg et al.

```

Setup: pick a curve  $G$  of prime order  $q$ ;
generator  $g$  keygen:  $sk \leftarrow \mathbb{Z}_q$ ,  $pk = g^{sk}$ 

Prove( $sk, x$ ):
     $h = H_1(pk, x)$            // hash-to-curve
     $\Gamma = h^{sk}$ 
     $y = H_2(\Gamma)$            // the random
output
     $(c, s) = \text{Schnorr-style NIZK proof}$ 
    return  $(y, \pi = (\Gamma, c, s))$ 
  
```

```

Verify( $pk, x, y, \pi$ ):
    check  $y == H_2(\Gamma)$  and NIZK accepts.
  
```

Why it's secure

Uniqueness, for fixed (sk, x) , only one valid y exists.

Assumption: discrete log hardness on G .

Pseudorandomness, without sk , y looks uniform in $\{0,1\}^\ell$.

Assumption: Decisional Diffie–Hellman (DDH).

Unforgeability, adversary can't produce π that verifies for a y they pick.

Assumption: NIZK soundness

Sortition, the math

VRF output $\rightarrow$ how many sub-votes does user i get?

```
// setup
W  = total stake in the network
τ  = target committee size (e.g. 2000)
p  = τ / W           // per-coin selection prob.
```

```
// user i with stake wi treats each coin
// as an independent Bernoulli(p) trial.
```

```
Let hash(y) / 2^ℓ → r ∈ [0, 1)
pick smallest j with
    r < Σk=0j B(k; wi, p)
```

$i \in \text{committee} \Leftrightarrow j \geq 1$, with j sub-votes

Consequences

$\mathbb{E}[\text{sub-votes}_i] = w_i \cdot p = w_i \cdot \tau / W$
 $\rightarrow$ stake-weighted, as intended.

$\mathbb{E}[\text{committee size}] = \sum_i w_i \cdot p = \tau$
 $\rightarrow$ hits the target on average.

$\text{Var}[\text{committee}] \approx \tau(1 - p) \approx \tau$
 $\rightarrow$ std-dev $\approx \sqrt{\tau} \approx 45$ when $\tau = 2000$.

Malicious sub-votes $\leq (h \cdot \tau) + O(\sqrt{\tau})$
 $\rightarrow$ Chernoff gives $\Pr[\text{fail}] \leq e^{-(\tau \epsilon^2/3)}$.

This is why the committee is stake-weighted AND size-controlled simultaneously, the VRF's uniform output plus the binomial CDF gives you both.

Why the committee must be SECRET

A public committee = a target list for the adversary.

✗ Public committee

Announce "validators for round 42 are Alice, Bob, Carol..."

Adversary reads list → DDoS their IPs, or bribes them (only 3 people to convince).

This is Ethereum's "proposer lookahead" problem, mitigated with slots but not eliminated.

✓ Secret self-selection

Every user runs VRF locally, silently.

You learn "I'm on the committee" the same moment you need to speak.

The moment you speak, the adversary learns it too, but your work is done. "Speak once, then disappear."

BA*, the protocol, at a glance

Every round = one proposer step + Byzantine Agreement on the block



Key invariant

Each step uses a **freshly sampled committee** (via VRF on that step's seed). If $> 2/3$ of sub-votes agree, the step terminates.

Player replacement:

even if the adversary corrupts a committee member right after they speak, they've already sent their message, harm already zero.

Step 1, Block Proposal

Who gets to suggest the next block?

- **Target proposer committee: expected size ≈ 20 users.**
- Every user i runs: $j = \text{sortition}(\text{VRF_sk}(\text{seed} \parallel \text{"Proposer"}), \text{stake_i}, p)$
- If $j > 0$, i is a "potential leader". Broadcasts a proposed block + VRF proof.
- **Tie-breaker: the proposer with the smallest hash(VRF_y) wins. This is canonical, verifiable.**
- Other users run the VRF check, gossip the proof + block.
- **End of Step 1: everyone "sees" ~ 20 candidate blocks; they agree on the min-hash one.**



Why does this work under adversary?

With ~ 20 expected proposers and $< 1/3$ adversarial stake, $\Pr[\text{at least one honest proposer}] \geq 1 - (1/3)^{20} \approx 1 - 3 \times 10^{-10}$.

Steps 2–3, Soft Vote & Certify Vote

This is where BFT happens, but only among the committee.

Step 2, Soft Vote

Goal: filter to one candidate block.

Fresh committee (expected ≈ 1000 users) runs VRF for role "Soft".

Each member:

- Collects proposed blocks
- Votes for the one with lowest VRF hash
- Signs with participation key

If $> 2/3$ sub-votes agree $\rightarrow$ block passes.
Else $\rightarrow$ empty block (EMPTY_B).

Step 3, Certify Vote

Goal: commit it, immutably.

Another fresh committee for role "Cert".

Each member:

- Checks block from Step 2
- Signs hash(block)
- Broadcasts cert vote

If $> 2/3$ sub-votes $\rightarrow$ **block is final.**

The $2/3$ signatures **are the certificate**, a light client needs only this.

Safety vs Liveness, Algorand's choice

CAP theorem is real. What does Algorand give up?

Every consensus protocol under partition must pick one:

Safety (Algorand's pick)

"Nothing bad happens." Honest nodes never commit conflicting blocks. If partitioned, the chain halts rather than forking.

Liveness (Bitcoin's pick)

"Something eventually happens." The chain always progresses, but may fork. You resolve with longest-chain.

Algorand's guarantees

- ✓ With synchronous network: both safety AND liveness, immediate finality.
- ✓ Under partition: safety always preserved, no conflicting commits.
- ⚠ Under partition: liveness may pause until network heals.

Key theorem (paper §4): "Forks last at most one round."

Committee size, the math (lightly)

How big is "big enough"?

Adversary holds fraction $h < 1/3$ of stake.

Committee size n drawn, Binomial(n, h) malicious.

Need: fraction malicious in committee $< 1/3$ with overwhelming probability.

Chernoff gives:

$$\Pr[\text{fail}] \leq \exp(-n \cdot \epsilon^2/3)$$

For $h = 0.2$, $\epsilon = 0.1$, $n = 2000$:

$$\Pr[\text{fail}] \approx 10^{-10}$$

Takeaways

- Algorand runs with committee ≈ 1000 – 4000 per step.
- Even with 1M users total, each step costs only ~ 1000 signatures, $O(1)$ per user.
- The tighter h is to $1/3$, the bigger n must be. Near $h \rightarrow 1/3$, no finite committee is safe.
- That's why the " $< 1/3$ adversarial stake" assumption is strict, not " $< 1/2$ ".
- Compare to PBFT: n = ALL validators. Algorand: n = tiny random sample.

What adversary is Algorand defending against?

Not a strawman, one of the strongest in BFT literature.

- **Adaptive corruption, adversary can corrupt any user at any time.**
 - But only after seeing messages, player-replacement nullifies post-hoc corruption.
- **Controls < 1/3 of total stake (not nodes, stake).**
- Can delay, reorder, drop messages in the gossip network.
- Cannot forge signatures or break VRF pseudorandomness.
- **Network assumption: eventually synchronous, messages delivered within Δ , after some GST.**
 - Weak synchrony: during brief bad periods, safety is preserved, liveness pauses.

This is strictly stronger than Casper FFG's static adversary model.

Why Algorand (almost) never forks

Probability of fork under synchrony: 10^{-18} , effectively zero

Bitcoin

Forks are normal.
Orphan blocks happen daily.
Finality = longest-chain
probabilistic.

Ethereum

Uncles + reorgs ≤ 2 blocks
common.
Casper FFG finalizes in epochs.
Short-range reorgs visible to
users.

Algorand

No forks under synchrony.
Under partition: fork lasts ≤ 1
round (paper theorem).
No reorgs, no chain reorg logic
in clients.

What it means for app developers

You can show the user "payment confirmed" the moment the block is certified, no "wait for 6 confirmations", no reorg defense.

Participation keys, forward security

A clever trick: keys that self-destruct after one use

- Spending key (cold): signs transactions. Kept offline.
- Participation key (hot): signs consensus votes. Online, exposed.
- **Each participation key is an ephemeral key pair, valid for one step only.**
 - User pre-generates a pool of μ ephemeral keys covering future rounds.
 - After using, the key is deleted, "honest amnesia".
- **Why? Prevents long-range attacks: an adversary who later steals your hot key cannot retroactively sign old blocks.**
- This is how Algorand resists the "nothing-at-stake" and "posterior corruption" problems that haunt naive PoS.

Compare: Ethereum's slashing punishes but doesn't prevent; Algorand makes the attack cryptographically impossible.

Consensus comparison, final scorecard

Bitcoin PoW · Ethereum Casper · PBFT · Algorand

	Bitcoin PoW	Ethereum PoS (Casper)	PBFT	Algorand BA*
Participant set	Anyone w/ ASIC	Staked validators	Known set	Everyone with ≥ 1 algo
Proposer selection	Hash lottery	RANDAO	Round-robin	VRF lottery (secret)
Voters	All miners	All validators	All (n)	Random committee
Msg complexity per block	Broadcast	$O(n)$	$O(n^2)$	$O(1)$ per user
Finality	6 blocks (~60m)	2 epochs (~13m)	Immediate	Immediate
Adversary tolerance	< 50% hash	< 33% stake	< 33%	< 33% stake (adaptive)
Forks possible?	Yes (common)	Yes (short reorgs)	No	No (under sync)

PART 2

Execution

What can you actually run on this thing?

Three virtual machines, quick refresher

Bitcoin Script · EVM · AVM

	Bitcoin Script	EVM	AVM (Algorand)
Paradigm	Stack, non-TC	Stack, Turing-complete	Stack, bounded (non-TC)
Language	Script opcodes	Solidity / Vyper → bytecode	PyTeal / Reach → TEAL
State	Stateless (UTXO locks)	Persistent world state	Both (stateless + stateful)
Loops	No	Yes (gas-metered)	Bounded (no unbounded loops)
Halting	Guaranteed (finite)	Gas-metered	Guaranteed (static bounds)
Typical use	Multi-sig, timelocks	Full dApps, DeFi, NFTs	ASAs, AMMs, on-chain logic

Note: the original 2017 paper says little about smart contracts, AVM came in 2020.

Wait, non-Turing-complete is a FEATURE?

Yes. And you already know why.

EVM (Turing-complete)

Halting problem says: you cannot statically decide if a program halts.

Workaround: gas.

Costs:

- Users must estimate gas (and fail txs)
- Gas golf → obfuscated code
- DoS via expensive contracts (The DAO)
- Reentrancy bugs (because programs can do anything)

AVM (bounded)

All programs provably halt. Static worst-case cost.

Wins:

- Predictable fees (flat 0.001 ALGO)
- No gas estimation, tx either valid or not
- Easier formal verification
- No reentrancy, subroutines can't recurse

Trade-off:

- Some algorithms harder to express
- Must refactor recursive contracts to bounded loops

Fee structure, compared:



Mint
NFT + pay
huge Ethereum
gas fees



Mint on
Algorand
with tiny fees

Two smart-contract modes

Stateless (approval) vs Stateful (application)

Stateless

A "smart signature" that approves/rejects a transaction.

Analogy: a **Bitcoin Script unlock condition**, but programmable.

Uses:

- Multi-sig
- Time-locked escrow
- Atomic swap HTLCs
- Delegated spending rules

No persistent storage.

Stateful (Application)

A program with global and per-account storage.

Analogy: an **Ethereum contract**, but bounded.

Storage tiers:

- Global state (app-wide)
- Local state (per opted-in account)
- Box storage (arbitrary KV, 2021+)

Uses: AMMs, DAOs, lending, voting.

TEAL, the bytecode

Transaction Execution Approval Language

```
// Simple TEAL: "approve if amount < 1000"
#pragma version 6

txn TypeEnum
int pay
==

txn Amount
int 1000
<

&& // both must hold
return
```

What you need to know

- Stack-based, Forth-like, familiar to anyone who's seen Bitcoin Script.
- Opcodes: arithmetic, comparison, crypto (ed25519, keccak), txn fields, state access.
- Written in PyTeal (Python DSL), Reach, Tealish, or directly.
- Bounded: loop depth $\leq$ fixed constant, subroutine recursion disallowed.
- Opcode budget per tx group: 700 by default; "opt-up" rooms let apps pool budget.
- Output: 0 (reject) or 1 (approve). No in-band value return.

First-class primitives: ASAs & atomic transfers

What ERC-20 and Uniswap atomicity hacks give you, but native.

Algorand Standard Assets (ASAs)

One transaction type creates a fungible or non-fungible token.

No contract to deploy, no approval pattern, no `allowance()` bugs.

Built-in features:

- Freeze, clawback, decimals
- Per-asset min balance (spam control)
- Opt-in before receiving (by design)

Compare: every ERC-20 is a fresh Solidity contract with its own bugs.

Atomic Transfers

Group up to 16 transactions. Either all commit, or none do.

Atomicity enforced by consensus, no escrow contract needed.

Enables:

- P2P atomic swaps (no HTLCs)
- Circular multi-party trades
- Flash-loan-style logic safely

Compare: on Ethereum you'd write a contract, pay deployment gas, audit it, and still risk reentrancy.

Execution, one-liner summary

Different design philosophies, not just different VMs

Bitcoin

"Money should be boring."

Script is intentionally weak.

No loops. No state.

Just: "can this coin be spent?"

Ethereum

"Computer on a chain."

Turing-complete EVM.

Maximum expressiveness.

Price: gas, bugs, reentrancy.

Algorand

"Safe by construction."

Bounded AVM.

Native ASAs, atomic groups.

Stateful+stateless split.

Harder to express. Easier to verify.

PART 3

State

How does a new node catch up? How is the ledger stored?

State model: accounts (like Ethereum)

Not UTXO. Algorand made the same choice as Ethereum here, but simpler.

- Every account has: address, balance, min-balance, asset holdings, local app state.
- A transaction mutates account balances directly, no UTXOs to sweep.
 - Why not UTXO? For smart contracts, global state is easier to reason about.
- **Min-balance requirement: 0.1 ALGO base + 0.1 per asset held + more per app opted-in.**
 - This is spam prevention, you can't bloat the state for free (unlike Ethereum pre-EIP-150).
- **Opt-in model: before receiving an ASA or engaging with an app, your account must opt in.**
 - Prevents "dusting" and unwanted assets, you own only what you've chosen to own.

What's in a block?

Algorand's block is not a Bitcoin block

```
Block_r = {  
  round:      r,  
  prev_hash:  H(Block_{r-1}),  
  txns:       [tx1, tx2, ...],  
  seed Q_r:   VRF of proposer,  
  proposer:   ℓ_r,  
  state_root: Merkle root after txns,  
  cert_r:     > 2/3 committee sigs  
}
```

Two important bits

Q_r (the seed), this is the random beacon that drives sortition for round r+1.

It's derived from the proposer's VRF output, so the next committee is as unpredictable as the previous proposer was random.

cert_r, the 2/3-committee signature bundle. This is what makes the block FINAL.

A light client doesn't need to re-verify txns, just verify cert_r against known participation keys.

How does a new node catch up?

Initial Block Download vs Snap Sync vs Catchpoints

	Bitcoin (IBD)	Ethereum (snap/Geth)	Algorand (Fast Catchup)
What you download	Every block from genesis	Block headers + state snapshot	Latest catchpoint + short history
Verification	Re-execute every tx	Verify snapshot against header root	Verify catchpoint cert (committee sigs)
Typical time	Days to weeks	Hours	Minutes (1–10)
Storage needed	~600 GB (full) / ~7 GB (pruned)	~1 TB full / ~500 GB snap	~30 GB (relay) / ~500 MB (participation)
Finality guarantee	Chain is longest at end	Header chain matches canonical	Every state ever seen is final

Catchpoints work because every Algorand state is certified, no "maybe this is canonical" ambiguity.

Me, waiting for my Bitcoin full node to sync:



Catchpoints, why this works on Algorand

The magic: final state = verifiable state

- Every ~10,000 rounds, the network publishes a "catchpoint": a Merkle root of the full state.
- New node downloads: latest catchpoint + block headers + a small number of recent blocks.
- **Verifies: catchpoint Merkle proof + headers' 2/3 participation-key signatures.**
 - Does NOT re-execute history, the signatures ARE the proof of correctness.
- Contrast with Bitcoin: "longest chain" means you must replay every block to know which one's longest.
- Contrast with Ethereum snap sync: you still need enough headers to trust the snapshot.
- **This is only possible because Algorand has immediate finality. No ambiguity → no replay.**

Node types in Algorand

Not all nodes carry the same load

Relay Nodes

- Gossip messages across the network
- High bandwidth, always on
- ~140 globally (Algorand Foundation-managed)
- Store full ledger

Participation Nodes

- Run the consensus protocol
- Hold ephemeral participation keys
- Any algo holder can run one
- Laptop-grade hardware is enough

Non-Participating

- Read-only, wallets, explorers
- Follow the chain, don't vote
- Can use archival or prune
- Light-client variants exist

Compare Ethereum: "node" means one thing. Algorand separates liveness (relays) from consensus (participation).

Honest trade-offs & criticisms

What Algorand gives up, and what people complain about

- **Economic centralization concern**
 - Algorand Foundation + early insiders held ~55% at launch → large stake concentration.
 - Distribution has improved since, but "< 1/3 adversarial" needs actually decentralized stake.
- **Liveness during partition**
 - Chain halts during partition (safety-first). PoW chains keep producing blocks, albeit forking.
 - For financial infrastructure, most would call this the right trade-off.
- **Less expressive smart contracts**
 - Bounded AVM makes some patterns awkward, e.g., recursive data structures.
- **Ecosystem is smaller than Ethereum's, fewer tools, fewer audited libraries.**
- **Cryptography is heavier to understand (VRFs, BA*), making the protocol harder to audit at a glance.**

What the 2017 paper leaves open

The paper is about consensus, not the full system

- Smart contracts, not in the paper. AVM + TEAL came in 2020 (v2 of the mainnet).
- State trees, the paper assumes "a ledger"; concrete Merkle trees were engineering later.
- Economic model, the paper treats stake as a given, doesn't model inflation or rewards.
- Network layer, "gossip" is assumed; production uses structured relay topology.
- Participation key rotation, spelled out in the paper but engineering burden was underestimated.
- **Data availability / sharding, not addressed; Algorand L1 scales up via block size + speed, not shards.**

This is what paper-presentation season is for, "here's the idea, here's what it became."

Where is Algorand today?

Numbers, roughly (as of 2024-25)

~4.5s

block time

~6,000

TPS (theoretical)

~30M

accounts

~2 bn

txs lifetime

Real-world use

- FIFA ticketing (2022 World Cup), first sports federation on a public chain.
- Marshall Islands SOV, sovereign digital currency project.
- El Salvador CBDC research partner.
- Carbon credit markets, real-estate tokenization, academic credential issuance.
- DeFi footprint modest vs Ethereum L2s, Algorand DeFi TVL ≈ \$100M–200M range.

Rough figures; check algorand.foundation for exact live numbers before your presentation.

TL;DR, remember these 5 things

If you forget the rest, keep these

1

Pure PoS

Every algo = one coin-vote. Holding = participating. No staking, no delegation.

2

**Cryptographic
Sortition**

VRF picks a secret, tiny committee each step. Adversary can't target, can't bribe, can't DoS.

3

**BA* + Player
Replacement**

> 2/3 honest committee → block finalized in one round. Fresh committee each step = adaptive-secure.

4

Safety > Liveness

No forks under synchrony. Under partition: chain halts, never diverges. Finality is instant.

5

Bounded AVM

TEAL programs halt statically. ASAs and atomic transfers as native primitives. Safe by construction.

References & further reading

If you want to dig deeper

Primary

- Chen, J. & Micali, S. *Algorand*. arXiv:1607.01341, the paper we read.
- Gilad, Y. et al. *Algorand: Scaling Byzantine Agreements for Cryptocurrencies*. SOSP 2017, the "systems" companion paper.

Background

- Micali, Rabin, Vadhan. *Verifiable Random Functions*. FOCS 1999.
- Castro & Liskov. *Practical Byzantine Fault Tolerance*. OSDI 1999, the BFT baseline.
- Buterin, Griffith. *Casper the Friendly Finality Gadget*. 2017, Ethereum's answer.

Code & docs

- github.com/algorand/go-algorand, reference implementation in Go.
- developer.algorand.org, TEAL spec, SDKs, tutorials.
- algorand.foundation, protocol specs, governance, live stats.

Questions?

"If a block falls in a forest and no committee certifies it..."

Thanks for listening, let's discuss.